

CPS572 Final Project: Multi-Task LLM Fine-Tuning

Aaron Huberman

Qavi

1. Methodology

1.1 Overview

This project fine-tunes Llama-3.1-8B via Low-Rank Adaptation (LoRA) to improve performance simultaneously on instruction following (IFEval), math reasoning (GSM8K), and code generation (HumanEval). All training used the Tinker API with supervised fine-tuning (SFT) as the primary method, followed by a reinforcement learning stage using a custom GRPO implementation.

1.2 Model and LoRA Configuration

Base model: `meta-llama/Llama-3.1-8B`. LoRA rank 128 applied to all layer types (attention, MLP, unembedding). Renderer: `role_colon`.

1.3 Training Datasets

Dataset	Task	Size	Notes
openai/gsm8k (train)	Math	7,473	All examples, CoT prefix
microsoft/orca-math-word-problems-200k	Math	50,000	Diverse GPT-4 math
meta-math/MetaMathQA	Math	50,000	GSM8K/MATH rewrites
allenai/tulu-3-sft-mixture	IF	100,000	General instruction following
argilla/ifeval-like-data (filtered)	IF	56,339	Verified IFEval-style examples
nvidia/OpenCodeInstruct	Code	100,000	Quality filtered ≥ 0.9

Weighted sampling: math 2.0, IF 4.0, code 2.0. Chain-of-thought prefixes added to all math responses.

1.4 Hyperparameters

Parameter	Value	Rationale
Learning rate	3e-5	Reduced from 5e-5 for continuation stability
Warmup steps	100–200	Linear warmup prevents instability at rank 128
Weight decay	0.01	L2 regularization for generalization
Grad clip norm	1.0	Prevents destabilizing gradient spikes
LoRA rank	128	Doubled from 64 for 8B multi-task capacity
Max length	1024	Captures full CoT reasoning chains
Total steps	16,000+	8,000 initial + 8,000 continuation

1.5 Training Procedure

Training ran in two phases. Phase 1: 8,000 steps from a randomly initialized LoRA adapter, checkpointing every 1,000 steps. Phase 2: resumed from the step-8000 checkpoint via `create_training_client_from_state` with `lr=3e-5` and a different random seed to sample fresh examples. Intermediate checkpoints were evaluated to identify the optimal stopping point.

2. Extensions

2.1 Constraint Failure Analysis and Targeted Dataset Selection

Rather than treating IFEval as monolithic, we performed programmatic constraint-level analysis on evaluation logs using the `instruction_following_eval` library. This revealed three failure buckets: case constraints (95–100% failure: `english_lowercase`, `english_capital`), length/count constraints (79–100% failure: `number_words`, `number_sentences`, `number_paragraphs`), and keyword constraints (81–84% failure: `forbidden_words`, `frequency`). Armed with this diagnosis, we identified `argilla/ifeval-like-data` (filtered): 56,339 examples generated by Qwen2.5-72B-Instruct and verified against the IFEval constraint checker. Replacing Tulu-3 with this dataset produced an 18-point IFEval jump (41.9% $\rightarrow$ 60.2%), the largest single improvement of the project.

2.2 Manual GRPO on an SFT-Only API

Tinker does not natively support RL, so we implemented GRPO using available primitives. Each step: (1) sample a prompt from argilla; (2) generate $G=16$ responses via `create_sampling_client` at temperature 1.0; (3) score each with partial credit reward (fraction of constraints passing); (4) compute group-relative advantages $A_i = (r_i - \bar{r})/(\sigma_r + \epsilon)$; (5) create weighted datums for positive-advantage outputs scaled by A_i ; (6) `forward_backward + optim_step`. Key engineering challenges included discovering that `build_generation_prompt` returns `ModelInput` directly, that `SampledSequence` exposes `tokens` not `text`, and that stop sequences prevent the model from generating fake multi-turn continuations. GRPO showed modest gains on GSM8K (+0.7%) and HumanEval (+0.6%) with no IFEval regression, demonstrating successful multi-task RL without catastrophic forgetting.

3. Results and Analysis

3.1 Progression Across Versions

Version	Model	IFEval	GSM8K	HumanEval	Key Change
Baseline	8B	22.8%	9.7%	0.6%	No fine-tuning
v1	8B	37.7%	51.3%	42.1%	Real data, 500 steps
v2	8B	41.9%	56.7%	46.3%	Orca-Math, Tulu 50k, CoT
v3	8B	38.9%	58.9%	45.1%	System prompt (regression)
v4	8B	60.2%	61.1%	48.2%	Argilla replaces Tulu
v4b+GRPO	8B	61.3%	64.5%	46.3%	300-step GRPO
v5 step 5000	8B	63.5%	63.7%	44.5%	Large dataset
v5 step 8000	8B	63.0%	68.3%	45.7%	Continued training

3.2 Intermediate Checkpoint Analysis

We evaluated checkpoints every 1,000 steps during v5 training. The loss curve showed continued decline throughout all 8,000 steps (avg loss: 0.422 at step 1000, 0.395 at step 5000, 0.364 at step 8000), with no sign of convergence or overfitting. GSM8K improved monotonically from 56.7% at step 3000 to 68.3% at step 8000. IFEval peaked at step 5000 (63.5%) and declined slightly at step 8000 (63.0%), suggesting modest task-specific overfitting for constraint following. HumanEval peaked at step 8000 (45.7%). This confirms the professor’s warning that the final checkpoint is not always optimal: for a competition submission maximizing average score, step 8000 is best overall, but a practitioner optimizing IFEval specifically should prefer step 5000. We used step 8000 as the submission checkpoint.

3.3 What Worked

The argilla dataset selection was by far the most impactful intervention. Scaling to 8B gave large gains on all tasks. Chain-of-thought prefixes improved GSM8K substantially. Continued training beyond 2,000 steps produced meaningful improvements — the loss curve had not converged, and scores improved monotonically through 8,000 steps.

3.4 What Did Not Work

Adding a system prompt to all Tulu examples (v3) caused a 3-point IFEval regression. Post-hoc constraint analysis confirmed it made every constraint category worse, likely because the system prompt caused the model to prepend constraint-checking preambles that violated prompt-level strict scoring. Generic Tulu data showed rapidly diminishing returns beyond 50k examples for mechanical constraint types — the model learned general conversational instruction following but could not generalize to verifiable surface constraints. GRPO produced limited IFEval improvement due to sparse reward signal: at 60%+ accuracy, most prompts yield identical rewards across all G samples, producing no gradient update.

3.5 Leaderboard

Task	Our Best	Competition Leader	Gap
IFEval	63.5%	72.5%	−9.0%
GSM8K	68.3%	71.5%	−3.2%
HumanEval	48.2%	59.4%	−11.2%

Supplementary material (LLM Usage, Tinker Feedback) follows on the next page.

Supplementary Material

LLM Usage

Claude (Anthropic, claude-sonnet-4-6) was used throughout this project. All LLM usage is disclosed below.

Code writing (LLM-assisted). Claude wrote the majority of code including: all versions of `evaluation/train_and_publish.py` (dataset loaders, weighted sampling, training loop, checkpoint saving logic); `evaluation/grpo_train.py` (the full GRPO implementation from scratch); and `test.py` (the constraint failure analysis script). Claude also debugged API errors throughout the project, including identifying the correct Tinker method signatures (`build_generation_prompt`, `save_state`, `SampledSequence.tokens`), fixing Python import errors, and resolving Windows-specific setup issues (SSH keys, PowerShell venv activation, symlink warnings).

Dataset research and selection (LLM-assisted, student-directed). Claude searched HuggingFace for datasets matching the diagnosed constraint failure modes, identifying `argilla/ifeval-like-data`, `microsoft/orca-math-word-problems-200k`, and `meta-math/MetaMathQA`. Claude streamed first examples to inspect schemas before writing loading functions. The decision to pursue a verified IFEval-specific dataset rather than generating synthetic data was made by the students on ethical grounds.

Experimental design (collaborative). Claude participated in decisions including model size selection, data mixing ratios, learning rate scheduling, and GRPO design. Claude provided technical explanations of GRPO mechanics, LoRA theory, and reward signal design. Key strategic decisions — rejecting synthetic data generation, identifying the constraint failure analysis approach, deciding to run overnight training runs — were made by the students.

Analysis and interpretation (collaborative). Claude helped interpret evaluation results and wrote the constraint failure analysis parsing script. The insight that the analysis should directly inform dataset selection rather than prompting strategies was driven by the students.

Report writing (LLM-assisted). This report was drafted with Claude’s assistance, based on a running `CHANGES.md` changelog and experiment results maintained by the students throughout the project. All factual claims reflect actual experimental outcomes.

Tinker Platform Feedback

Hardest part. The gap between the SFT-oriented API and advanced training methods. Implementing GRPO required significant reverse-engineering: discovering that `save_state` (not `save_weights`) is needed for resumable checkpoints, that sampler weights and training weights use different path formats, and that `create_training_client_from_state` does not accept sampler paths. Better documentation of the checkpoint path format taxonomy would have saved considerable debugging time. Windows compatibility was also a persistent issue — SSH key setup, HuggingFace Hub symlink warnings, and PowerShell differences from bash all required undocumented workarounds.

Tinker-cookbook usage. Yes, extensively. The cookbook was essential for conversation rendering (`get_renderer`, `build_generation_prompt`), tokenization (`get_tokenizer`), and datum construction (`conversation_to_datum`, `TrainOnWhat`). What was missing was documentation on `build_generation_prompt`’s return type — it returns a `ModelInput` directly rather than tokens, which was not obvious and caused the GRPO prompt rendering to fail initially. Type annotations or a short docstring would have helped.

One improvement. Native GRPO or PPO support. The verifiable reward paradigm is a natural fit for Tinker’s API — the platform already has all the necessary primitives but combining

them for RL requires substantial undocumented engineering. A `grpo_training_client` analogous to `create_lora_training_client` that handles the sampling loop, advantage computation, and reference model KL penalty internally would dramatically lower the barrier to RL experimentation, which the project documentation explicitly encourages.

Would you use Tinker again? Yes. The ability to run large-scale 8B training jobs through a simple Python API without managing GPU infrastructure was genuinely valuable. The main improvement needed is documentation depth — the API surface is larger than what the cookbook covers, and discovering available methods required exploratory debugging via `dir()` and `inspect.signature()`.